Task 1: QA & Debug Report

Candidate: Arbaj Ahmad

Role: Automation & QA Developer (take-home assessment)

Application tested: OWASP Juice Shop, official online demo (intentionally vulnerable test app) https://juice-shop.herokuapp.com/

1. Scope and approach

I exercised the main user flows (registration, login, search, browsing) and then attacked them with edge cases: special characters, script snippets, guessed URLs and invalid paths. Testing was done manually in the browser only, on an app that is designed for security practice. Findings are ordered by severity.

2. Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Login bypass via SQL injection: attacker can log in as admin without a password**	1. Open the app and go to Account > Login. 2. In the Email field enter: ' or 1=1-- 3. Enter any text as the password (e.g. x). 4. Click Log in.	**Expected:** Login is rejected with an "Invalid email or password" message. **Actual:** Login succeeds and the session belongs to the first user in the database, the administrator account.	**Critical**	User input is concatenated directly into the SQL query instead of using parameterized queries, so the quote closes the string and the rest of the query is commented out.
2	**Search box is vulnerable to DOM-based XSS: script runs from typed text**	1. On the home page click the search (magnifier) icon. 2. Type: <iframe src="javascript:alert(`xss`)"> 3. Press Enter. 4. Observe the result page.	**Expected:** The text is shown as harmless plain text, or no results are found. **Actual:** The browser executes the injected code and an alert box appears.	**High**	The search term is inserted into the page as raw HTML (input is trusted and not encoded/sanitized before rendering).
3	**Server exposes a public /ftp directory listing containing internal documents**	1. In the browser address bar append /ftp to the app URL and press Enter. 2. Open any listed file, e.g. a .md or .bak document.	**Expected:** Access is denied (403/404) because the folder is not meant for the public. **Actual:** A full directory listing is displayed and internal files can be downloaded without logging in.	**High**	Directory listing is enabled on a folder that is served statically with no authentication or access control.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	**Unhandled server error page leaks technical details**	1. In the address bar, open a made-up API path such as /rest/qwertz. 2. Observe the response page.	**Expected:** A short, friendly error message with no internal information (or a clean 404). **Actual:** A raw error page with a stack trace is shown, revealing server framework and file paths.	**Medium**	The default error handler is left in development mode and returns the full error object instead of a generic message.
5	**Registration accepts very weak passwords**	1. Go to Account > Login > "Not yet a customer?" to open registration. 2. Enter a new email, the password 12345, and the same value in Repeat Password. 3. Choose a security question and answer, then click Register.	**Expected:** The form rejects the password and explains the strength rules (length, mix of characters). **Actual:** The account is created with a trivially guessable password.	**Medium**	Password validation only checks a very short minimum length and has no complexity or common-password check.